

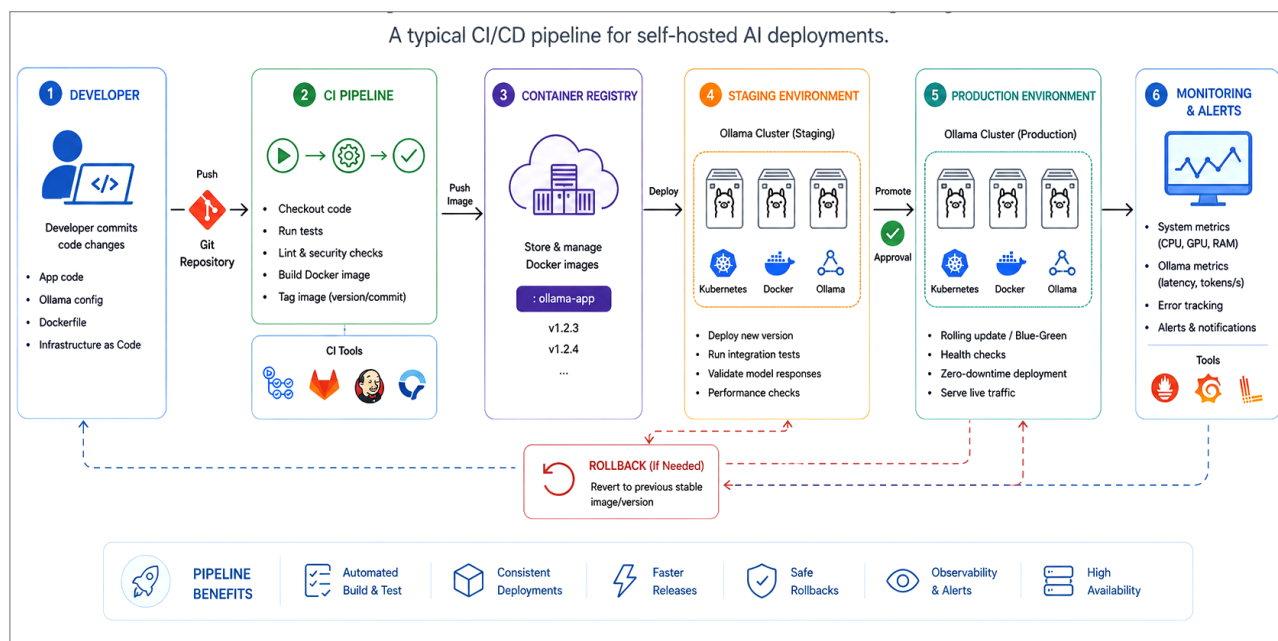


Figure 3: CI/CD pipeline for self-hosted AI deployments

How everything fits together

The architecture is not complicated. That is the whole point.

Your apps talk to Ollama through HTTP. Ollama loads the model into memory, runs it, and sends the answer back. Docker manages the lifecycle. That is basically it.

In a real client setup, the path goes something like this. Request comes in. Hits an Nginx or Traefik proxy. Lands at the app backend container. Backend builds the prompt, sometimes pulling extra context from a vector database like Qdrant or Weaviate. It calls Ollama on the internal Docker network. Ollama runs the model. The backend cleans up the response. The user gets their answer. Everything stays inside the customer network. Nothing crosses out.

Monitoring uses regular tools. Prometheus for metrics. Loki for logs. Grafana to see what is happening. No exotic stack. Which is why it works.

Where this is being used right now

A few patterns keep showing up in our projects.

Internal knowledge bots are the most common. Companies plug Ollama into a retrieval setup so their staff can ask questions about internal wikis, HR policies, contracts, technical docs. Sensitive stuff never leaves the office.

Code helpers are next. Engineering teams hook up code-focused models like CodeLlama or DeepSeek-Coder to their IDEs and CI pipelines. Their proprietary code never sees an external API.

Customer support drafting is big in BFSI. Banks use private AI to suggest reply drafts to their support agents. Faster handle times. No customer information leaving the network.

Document work is huge in legal and consulting firms. Summarising long files, finding clauses, answering questions about case papers. All local.

Edge deployments are the newest one. A manufacturing client of ours runs Phi-3 on devices on their factory floor. AI that works even when the internet does not.

The Ollama and Docker combination keeps hitting the same sweet spot. Simple enough to maintain. Serious enough to deploy.

The honest pros and cons

I will be straight with you here.

The wins are real. You control your data. Your costs are predictable. You can fine-tune and customise as much as you want. You are not stuck with one vendor's pricing whims. For teams with steady AI usage, the total cost over a year almost always beats paying per token.

The trade-offs are also real. Open models are good but still trail the very best paid models on certain hard tasks. Complex reasoning. Very long context. Tricky math. If your app desperately needs top-tier quality on subtle work, self-hosting may not be the right answer for that specific piece. Hardware costs upfront. A 70B parameter model needs serious GPU memory, often multiple A100s or H100s. That money has to come from somewhere. And the operations work shifts to your team. Patches, upgrades, monitoring, capacity planning. That is now your job.

So no, this is not a magic replacement for paid APIs. It is a strong option that fits a lot of use cases, sometimes better than the alternatives. Your job is to figure out where your workload fits.